ASESII 24A02 Project-Based Quiz 1

Ridge, Lasso, and Regularization for Neural Features

Group 04
Nguyen Thi Yen Nhi (24125071), Nguyen Khang Thinh (24125104)
Nguyen Van Tinh (24125106), Nguyen Le Quoc Huy (24125100)

2026-07-14

Contents

Authorship and contribution statement

We confirm that every group member can explain the submitted analysis. AI use is reported in `Group04_AI_Log.csv`, and the group checked every AI-assisted item used in this report.

Student	ID	Main contributions	Verification performed
Nguyen Thi Yen Nhi	24125071	[Work]	[Check]
Nguyen Khang Thinh	24125104	[Work]	[Check]
Nguyen Van Tinh	24125106	[Work]	[Check]
Nguyen Le Quoc Huy	24125100	[Work]	[Check]

Abstract

Maximum 150 words: prediction problem, methods, main evidence, and recommendation.

Reproducible setup

```
required_packages <- c("tidyverse", "glmnet", "broom", "knitr")
missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]
if (length(missing_packages)) {
  stop("Install the required packages before rendering: ",
       paste(missing_packages, collapse = ", "))
}

library(tidyverse)
library(glmnet)
library(broom)
library(knitr)

# `params` exists only while knitting; the fallback
# keeps interactive chunk runs working.
has_params <- exists("params") && !is.null(params$group_number)
group_number <- if (has_params) as.integer(params$group_number) else 4L
if (length(group_number) != 1L || is.na(group_number) || group_number < 1L) {
  stop("Set params$group_number to your assigned positive group number.")
}
```

```r
seeds <- list(
  split = 240200L + group_number,
  cv = 240300L + group_number,
  features = 240400L + group_number
)
```

Shared helper functions

```r
find_exam_data <- function(filename) {
  input <- knitr::current_input(dir = TRUE)
  input_dir <- if (length(input) && nzchar(input)) dirname(input) else getwd()
  candidates <- c(
    file.path(input_dir, "data", filename),
    file.path(input_dir, "..", "data", filename),
    file.path(getwd(), "data", filename)
  )
  existing <- candidates[file.exists(candidates)]
  if (!length(existing)) stop("Cannot locate ", filename, " by a relative path.")
  hits <- unique(normalizePath(existing, winslash = "/", mustWork = TRUE))
  if (length(hits) > 1L && length(unique(unname(tools::md5sum(hits)))) > 1L) {
    stop("Multiple nonidentical copies of ", filename, " were found.")
  }
  hits[[1L]]
}

split_rows <- function(n, train_prop = 0.80, seed) {
  stopifnot(n >= 10L, train_prop > 0, train_prop < 1)
  set.seed(seed)
  train <- sort(sample.int(n, floor(train_prop * n), replace = FALSE))
  list(train = train, test = setdiff(seq_len(n), train))
}

fit_scaler <- function(x, tol = 1e-12) {
  x <- as.matrix(x)
  center <- colMeans(x)
  centered <- sweep(x, 2L, center, "-")
  scale <- sqrt(colMeans(centered^2))
  keep <- is.finite(scale) & scale > tol
  if (!any(keep)) stop("No nonconstant training predictor remains.")
  list(
    center = center[keep],
    scale = scale[keep],
    keep = keep,
    dropped = colnames(x)[!keep]
  )
}
```

```r
apply_scaler <- function(x, prep) {
  x <- as.matrix(x)[, prep$keep, drop = FALSE]
  x <- sweep(x, 2L, prep$center, "-")
  sweep(x, 2L, prep$scale, "/")
}

make_foldid <- function(n, k = 5L, seed) {
  if (k < 3L || k > n) stop("Require 3 <= k <= number of training rows.")
  set.seed(seed)
  sample(rep(seq_len(k), length.out = n))
}

fit_cv_glmnet <- function(x, y, alpha, foldid) {
  glmnet::cv.glmnet(
    x = x,
    y = y,
    family = "gaussian",
    alpha = alpha,
    foldid = foldid,
    standardize = FALSE,
    intercept = TRUE,
    type.measure = "mse"
  )
}

score_regression <- function(y, prediction) {
  y <- as.numeric(y)
  prediction <- as.numeric(prediction)
  if (length(y) != length(prediction) || any(!is.finite(c(y, prediction)))) {
    stop("Metrics require equal-length finite vectors.")
  }
  c(RMSE = sqrt(mean((y - prediction)^2)),
    MAE = mean(abs(y - prediction)))
}

count_nonzero <- function(fit, s = "lambda.min", tol = 1e-8) {
  b <- as.matrix(stats::coef(fit, s = s))
  slopes <- b[rownames(b) != "(Intercept)", 1L]
  sum(abs(slopes) > tol)
}

safe_condition_numbers <- function(x, lambda, tol = 1e-10) {
  eigenvalues <- eigen(
    crossprod(x) / nrow(x),
    symmetric = TRUE,
    only.values = TRUE
  )$values
  largest <- max(eigenvalues)
```

```
  smallest <- max(min(eigenvalues), 0)
  cutoff <- tol * max(1, largest)
  c(
    gram = if (smallest <= cutoff) Inf else largest / smallest,
    ridge = (largest + lambda) / (smallest + lambda)
  )
}
```

1 Problem 1. Prediction design and leakage control

1.1 1A. Target and predictors

```
config <- list(
  file = "prostate.csv",
  response = "lpsa",
  excluded = "lpsa"
)
data_raw <- read.csv(find_exam_data(config$file), check.names = FALSE)
predictors <- setdiff(names(data_raw), config$excluded)
analysis_data <- data_raw[, c(config$response, predictors), drop = FALSE]

if (anyNA(analysis_data)) stop("Declare a training-only missing-value plan.")
if (!all(vapply(analysis_data, is.numeric, logical(1)))) {
  stop("The assigned response and predictors must be numeric.")
}
```

```
meaning <- c(
  lcavol  = "Log cancer volume",
  lweight = "Log prostate weight",
  age     = "Age at diagnosis",
  lbph    = "Log BPH amount",
  svi     = "Seminal vesicle invasion",
  lcp     = "Log capsular penetration",
  gleason = "Gleason score",
  pgg45   = "Percent Gleason grade 4/5",
  lpsa    = "Log prostate-specific antigen"
)
unit <- c(
  lcavol = "log cm3", lweight = "log g",   age   = "years",
  lbph   = "log cm2", svi     = "0/1",     lcp   = "log cm",
  gleason = "6 to 9", pgg45   = "percent", lpsa  = "log ng/mL"
)

data_overview <- tibble(
  Variable = names(data_raw),
  Role     = ifelse(names(data_raw) == config$response, "Response", "Predictor"),
  Meaning  = unname(meaning[names(data_raw)]),
```

```
  Unit     = unname(unit[names(data_raw)]),
  Type     = vapply(data_raw, function(x) class(x)[1L], character(1)),
  Distinct = vapply(data_raw, dplyr::n_distinct, integer(1), na.rm = TRUE),
  Missing  = vapply(data_raw, function(x) sum(is.na(x)), integer(1))
)

kable(
  data_overview,
  caption = paste("Response and candidate predictors, with the integrity checks",
                  "run before the split.")
)
```

Table 2: Response and candidate predictors, with the integrity checks run before the split.

Variable	Role	Meaning	Unit	Type	Distinct	Missing
lcavol	Predictor	Log cancer volume	log cm3	numeric	93	0
lweight	Predictor	Log prostate weight	log g	numeric	88	0
age	Predictor	Age at diagnosis	years	integer	31	0
lbph	Predictor	Log BPH amount	log cm2	numeric	42	0
svi	Predictor	Seminal vesicle invasion	0/1	integer	2	0
lcp	Predictor	Log capsular penetration	log cm	numeric	30	0
gleason	Predictor	Gleason score	6 to 9	integer	4	0
pgg45	Predictor	Percent Gleason grade 4/5	percent	integer	19	0
lpsa	Response	Log prostate-specific antigen	log ng/mL	numeric	85	0

```
n_duplicate_rows <- sum(duplicated(analysis_data))
```

Define the prediction task and justify every exclusion.

Reading the table. The file holds 97 patients and 9 columns: one response and 8 candidate predictors. Three integrity results clear the way for the split. Every column is numeric, so the design matrix needs no factor encoding and the template's type guard passes. No cell is missing (`Missing` is zero throughout), so no imputation is performed; the missing-value rule is still declared in 1B as a contingency, because a rule invented after seeing the data would be a decision made with holdout information. And 0 rows are duplicated, so no patient is silently counted twice, which would otherwise inflate the apparent sample size and let the same record fall on both sides of the split.

(`BPH` in the table abbreviates benign prostatic hyperplasia, the non-cancerous prostate enlargement recorded by `lbph`.)

The `Distinct` column carries the one substantive surprise. Two predictors are numeric but not continuous: `svi` takes only 2 values (it is a binary indicator of seminal vesicle invasion) and `gleason` takes only 4 (an ordinal biopsy score from 6 to 9). We keep both on their numeric scale, which is the standard `glmnet` convention, but their coefficients must later be read with that discreteness in mind rather than as smooth slopes. The `Unit` column exposes the second issue: the predictors live

on wildly different scales, from `pgg45` measured in percent (0 to 100) to several variables already on a log scale. Penalized regression shrinks coefficients toward zero, so a predictor measured in large units would be punished merely for its units. This is precisely why the design matrix is standardized in 1B, using constants computed on the training rows only.

Assignment. Group 04 is even-numbered, so the assigned dataset is `prostate.csv` and the assigned response is `lpsa`.

Response. `lpsa` is the natural logarithm of prostate-specific antigen (ng/mL), a continuous serum marker measured before surgery. It is modelled on the log scale, so every coefficient is read as a change in log PSA per one standard deviation of a standardized predictor.

Unit of observation and study population. One row is one male patient with clinically localized prostate cancer who underwent radical prostatectomy in the study of Stamey et al. (1989). The analysis file contains 97 complete patients. Any conclusion therefore generalizes only to comparable prostatectomy candidates, not to unscreened men in the general population.

Prediction objective. Predict pre-operative `lpsa` from routinely available clinical and biopsy measurements, and compare how OLS, ridge, and lasso trade prediction error against coefficient stability and interpretability when several predictors are correlated.

Candidate predictors. All eight remaining variables are retained: `lcavol`, `lweight`, `age`, `lbph`, `svi`, `lcp`, `gleason`, and `pgg45` (see the data dictionary). All are numeric, so the template's type guard passes. Two deserve comment: `svi` is binary (0/1) and `gleason` is an ordinal score taking values 6-9. We keep both on their numeric scale, which is the standard `glmnet` convention; standardizing them makes their coefficients comparable in magnitude to the continuous predictors, at the cost of interpreting a "one standard deviation" change in a binary variable.

Exclusions. The project brief requires only that `lpsa` itself be removed from the predictor set, and permits a further exclusion only when a statistical reason is documented. We document none, so exactly one variable is excluded: the response. No candidate predictor is computed from `lpsa`: all eight are measurements taken before surgery (tumour volume, prostate weight, age, benign hyperplasia, seminal vesicle invasion, capsular penetration, and the two Gleason summaries), and none uses PSA information in its definition.

Why an outcome-derived predictor would invalidate the comparison. A variable computed from the response carries the answer inside the design matrix. Its least-squares coefficient would absorb nearly all explained variance, the training and cross-validation errors of every method would collapse toward zero, and the resulting model would fail on any new patient whose PSA is not yet known – which is precisely the deployment situation. The comparison itself would also become meaningless: OLS, ridge, and lasso would all be fitting the same leaked signal, so differences between them would reflect how each penalty shrinks that one dominant column rather than how each method handles genuine multicollinearity among clinical predictors. The parallel assignment makes the point concrete: for `fat.csv` the brief excludes `siri`, `density`, and `free` because they are alternative body-fat calculations. `prostate.csv` has no such variable, which is why no additional exclusion is warranted here.

1.2 1B. Split and train-only preprocessing

```
split <- split_rows(nrow(analysis_data), seed = seeds$split)
train_id <- split$train
```

```r
test_id <- split$test

x_raw <- as.matrix(analysis_data[, predictors, drop = FALSE])
y_raw <- analysis_data[[config$response]]

x_prep <- fit_scaler(x_raw[train_id, , drop = FALSE])
x_train <- apply_scaler(x_raw[train_id, , drop = FALSE], x_prep)
x_test <- apply_scaler(x_raw[test_id, , drop = FALSE], x_prep)
y_train <- y_raw[train_id]
y_test <- y_raw[test_id]

foldid <- make_foldid(nrow(x_train), k = 5L, seed = seeds$cv)
```

Report seeds, dimensions, scaling, folds, and leakage safeguards.

1.3 1C. Training-data exploration

```r
# Use analysis_data[train_id, ] only. Add focused plots and summaries.
```

Interpret multicollinearity, one anomaly, and the question regularization will address.

2 Problem 2. OLS, ridge, and lasso

2.1 2A. OLS baseline

```r
# Fit OLS on x_train and y_train. Do not access y_test here.
```

2.2 2B. Ridge

```r
# Use fit_cv_glmnet(..., alpha = 0, foldid = foldid).
# Report lambda.min, lambda.1se, paths, coefficients, and CV evidence.
```

2.3 2C. Lasso

```r
# Use fit_cv_glmnet(..., alpha = 1, foldid = foldid).
# Report lambda.min, lambda.1se, paths, and nonzero predictors.
```

2.4 2D. Training-based comparison and locked core model

```r
# Build a comparison from training/CV quantities only.
```

Core model nominated before holdout evaluation: [method and reason]

2.5 2E. Perturbation or stability check

Prediction before running the check: [prediction]

```
# Implement one allowed training-only stability check.
```

Observed result and explanation: [result]

3 Problem 3. Mathematical mechanisms

3.1 3A. Ridge and conditioning

Write the ridge derivation. Define dimensions and explain uniqueness.

```
# Use x_train and the fitted ridge lambda.min.
# safe_condition_numbers(x_train, lambda = ...)
```

3.2 3B. Lasso optimality and soft thresholding

Derive the zero/nonzero optimality conditions and the orthonormal-design formula.

3.3 3C. Connect algebra to evidence

Connect one specific numerical result to Problem 3A or 3B.

4 Problem 4. Elastic net and a neural-feature bridge

4.1 4A. Research question and source map

Research direction: [weight decay / sparse weights / pruning / dropout]

Claim	Exact primary source and locator	What it establishes	What it does not establish
[Claim]	[Citation, section/page/equation]	[Support]	[Boundary]

4.2 4B. Elastic net on original predictors

```
alpha_grid <- c(0.10, 0.25, 0.50, 0.75, 0.90)

# Fit one cv.glmnet object per alpha with the same foldid.
# Select alpha and lambda using training CV only.
```

4.3 4C. Fixed ReLU features

```
relu <- function(z) pmax(z, 0)
M <- 200L
```

```
# TODO:
# 1. Generate A and bias once from seeds$features using the stated distributions.
# 2. Apply the same A and bias to x_train and x_test.
# 3. Fit a scaler to H_train only and apply it to H_test.
# 4. Fit ridge, lasso, and elastic net on H_train with the shared foldid.
```

State which weights are fixed, which are learned, and how active features are counted.

4.4 4D. Locked declaration and final holdout evaluation

Locked before viewing `y_test`: [preprocessing, candidate specifications, selected tuning values, feature seed, nominated deployment model, metrics]

```
# This must be the first analysis chunk that uses y_test for evaluation.
# Generate all fixed candidate predictions and one RMSE/MAE table here.
```

State whether the holdout evidence supports the predeclared choice. Do not retune.

4.5 4E. Deep-learning connection and limitations

Distinguish penalty, weight decay, output-weight sparsity, pruning, dropout, and a trained deep network. Cite the sources from 4A and discuss at least two limitations.

5 Problem 5. Report quality and reproducibility

5.1 5A. Reproduction record

```
1. Open a clean R session.
2. [Exact commands and folder layout.]
3. Render GroupXX_ProjectQuiz1.Rmd.
```

5.2 5B. Recommendation and limitations

Give the final recommendation, distinguish prediction from interpretation, and state limitations.

5.3 5C. AI verification record

AI-assisted item	How it was checked	Evidence	Modification or rejection
[Item]	[Method]	[Test/source/output]	[Decision]

Preflight and session information

```
# Add every object required to reproduce the final table and figures.
required_objects <- c(
  "train_id", "test_id", "x_train", "x_test", "y_train", "y_test", "foldid"
```

```
)
missing_objects <- setdiff(required_objects, ls())
if (length(missing_objects)) {
  stop(
    "Missing required objects: ",
    paste(missing_objects, collapse = ", ")
  )
}
stopifnot(length(intersect(train_id, test_id)) == 0L)
```

```
sessionInfo()
```

```
## R version 4.6.0 (2026-04-24 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Vietnamese_Vietnam.utf8  LC_CTYPE=Vietnamese_Vietnam.utf8
## [3] LC_MONETARY=Vietnamese_Vietnam.utf8 LC_NUMERIC=C
## [5] LC_TIME=Vietnamese_Vietnam.utf8
##
## time zone: Asia/Saigon
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] knitr_1.51      broom_1.0.13    glmnet_5.0      Matrix_1.7-5
##  [5] lubridate_1.9.5 forcats_1.0.1   stringr_1.6.0   dplyr_1.2.1
##  [9] purrr_1.2.2     readr_2.2.0     tidyr_1.3.2     tibble_3.3.1
## [13] ggplot2_4.0.3   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] generics_0.1.4     shape_1.4.6.1      stringi_1.8.7      lattice_0.22-9
##  [5] hms_1.1.4          digest_0.6.39      magrittr_2.0.5     timechange_0.4.0
##  [9] evaluate_1.0.5     grid_4.6.0         RColorBrewer_1.1-3 iterators_1.0.14
## [13] fastmap_1.2.0      foreach_1.5.2      backports_1.5.1    survival_3.8-6
## [17] scales_1.4.0       codetools_0.2-20   cli_3.6.6          rlang_1.2.0
## [21] splines_4.6.0      withr_3.0.2        yaml_2.3.12        otel_0.2.0
## [25] tools_4.6.0        tzdb_0.5.0         vctrs_0.7.3        R6_2.6.1
## [29] lifecycle_1.0.5    pkgconfig_2.0.3    pillar_1.11.1      gtable_0.3.6
## [33] glue_1.8.1         Rcpp_1.1.2         xfun_0.57          tidyselect_1.2.1
## [37] rstudioapi_0.18.0  farver_2.1.2       htmltools_0.5.9    rmarkdown_2.31
## [41] compiler_4.6.0     S7_0.2.2
```

Stamey, Thomas A., John N. Kabalin, John E. McNeal, et al. 1989. "Prostate Specific Antigen in the Diagnosis and Treatment of Adenocarcinoma of the Prostate. II. Radical Prostatectomy Treated Patients." *The Journal of Urology* 141 (5): 1076–83. https://doi.org/10.1016/S0022-5347(17)41175-X.